Phase 5: The Climax – Squashing and Achieving FHE

Fully Homomorphic Encryption over the Integers

Paper Presentation

April 14, 2026

Outline: Phase 5

1. The Depth Problem
2. Squashing the Decryption Circuit
3. The Shallow Circuit & Bootstrapping Achieved
4. References

The Crisis of Bootstrapping

Motivation

We established in Phase 1 that to achieve Fully Homomorphic Encryption, our scheme must be **bootstrappable**. *It must be able to homomorphically evaluate its own decryption circuit before the noise exceeds the $p/2$ boundary.*

The Crisis of Bootstrapping

Motivation

We established in Phase 1 that to achieve Fully Homomorphic Encryption, our scheme must be **bootstrappable**. *It must be able to homomorphically evaluate its own decryption circuit before the noise exceeds the $p/2$ boundary.*

Let us look at the mathematics of our decryption equation:

$$m' = (c \pmod{p}) \pmod{2}$$

The Crisis of Bootstrapping

Motivation

We established in Phase 1 that to achieve Fully Homomorphic Encryption, our scheme must be **bootstrappable**. *It must be able to homomorphically evaluate its own decryption circuit before the noise exceeds the $p/2$ boundary.*

Let us look at the mathematics of our decryption equation:

$$m' = (c \pmod{p}) \pmod{2}$$

To evaluate this homomorphically as a boolean circuit, we must compute it over the integers:

$$c \pmod{p} = c - p\lfloor c/p \rceil$$

The Wall of Integer Division

The Roadblock: To evaluate $c - p\lfloor c/p \rceil$, the circuit must compute $\lfloor c/p \rceil$. This requires **integer division**.

The Wall of Integer Division

The Roadblock: To evaluate $c - p\lfloor c/p \rceil$, the circuit must compute $\lfloor c/p \rceil$. This requires **integer division**.

When translated into a boolean polynomial circuit, division has a massive multiplicative depth.

The Wall of Integer Division

The Roadblock: To evaluate $c - p\lfloor c/p \rceil$, the circuit must compute $\lfloor c/p \rceil$. This requires **integer division**.

When translated into a boolean polynomial circuit, division has a massive multiplicative depth.

Recall our permitted polynomial degree bound d from Phase 2:

$$d \leq \frac{\eta - 4 - \log ||\hat{f}||_1}{\rho' + 2}$$

The Wall of Integer Division

The Roadblock: To evaluate $c - p\lfloor c/p \rceil$, the circuit must compute $\lfloor c/p \rceil$. This requires **integer division**.

When translated into a boolean polynomial circuit, division has a massive multiplicative depth.

Recall our permitted polynomial degree bound d from Phase 2:

$$d \leq \frac{\eta - 4 - \log ||\hat{f}||_1}{\rho' + 2}$$

The Reality: The standard division circuit's degree far exceeds this permitted noise budget. The decryption circuit is simply too deep. We must **"squash"** it.

Squashing: Shifting the Burden

To make the scheme bootstrappable, we must reduce the multiplicative depth of the decryption circuit.

Squashing: Shifting the Burden

To make the scheme bootstrappable, we must reduce the multiplicative depth of the decryption circuit.

Intuition (The Squashing Strategy)

Division by p is mathematically expensive. Multiplication is relatively cheap. We will shift the heavy computational lifting from the Decrypt algorithm (which is strictly constrained by the noise budget) to the Evaluate algorithm (which can be done in the clear by the server before the final homomorphic step).

Squashing: Shifting the Burden

To make the scheme bootstrappable, we must reduce the multiplicative depth of the decryption circuit.

Intuition (The Squashing Strategy)

Division by p is mathematically expensive. Multiplication is relatively cheap. We will shift the heavy computational lifting from the Decrypt algorithm (which is strictly constrained by the noise budget) to the Evaluate algorithm (which can be done in the clear by the server before the final homomorphic step).

New Parameters:

- κ: Precision of rational numbers.
- Θ: Size of a massive public set of rational numbers.
- θ: The Hamming weight of a sparse secret subset.

Modifying Key Generation: The Hint

During KeyGen, alongside p and pk^*, we generate a "hint" about the value of $1/p$.

Modifying Key Generation: The Hint

During KeyGen, alongside p and pk^*, we generate a "hint" about the value of $1/p$.

1. The New Secret Key ($\vec{s}$): Choose a random Θ-bit vector $\vec{s} = \langle s_1, \ldots, s_\Theta \rangle$ with a very low Hamming weight θ. Let $S = \{i : s_i = 1\}$. This sparse vector $\vec{s}$ becomes the new secret key.

Modifying Key Generation: The Hint

During KeyGen, alongside p and pk^*, we generate a "hint" about the value of $1/p$.

1. The New Secret Key ($\vec{s}$): Choose a random Θ-bit vector $\vec{s} = \langle s_1, \ldots, s_\Theta \rangle$ with a very low Hamming weight θ. Let $S = \{i : s_i = 1\}$. This sparse vector $\vec{s}$ becomes the new secret key.

2. The Public Rational Numbers ($\vec{y}$): Generate a set $\vec{y} = \{y_1, \ldots, y_\Theta\}$ of rational numbers in $[0, 2)$ with κ bits of precision, such that the subset S sums to approximately $1/p$:

$$\sum_{i \in S} y_i \approx \frac{1}{p} \pmod 2$$

Specifically, $\left[\sum_{i \in S} y_i\right]_2 = \frac{1}{p} - \Delta_p$, where $|\Delta_p| < 2^{-\kappa}$.

Modifying Evaluate and Decrypt

When the server finishes homomorphically evaluating a circuit, it holds a noisy ciphertext c^*. **Before returning it**, the server does post-processing in the clear.

Modifying Evaluate and Decrypt

When the server finishes homomorphically evaluating a circuit, it holds a noisy ciphertext c^*. **Before returning it**, the server does post-processing in the clear.

Server Post-Processing: For $i \in \{1, \ldots, \Theta\}$, the server computes:

$$z_i = [c^* \cdot y_i]_2$$

keeping only $n = \lceil \log \theta \rceil + 3$ bits of precision. The server outputs c^* and $\vec{z} = \langle z_1, \ldots, z_\Theta \rangle$.

Modifying Evaluate and Decrypt

When the server finishes homomorphically evaluating a circuit, it holds a noisy ciphertext c^*. **Before returning it**, the server does post-processing in the clear.

Server Post-Processing: For $i \in \{1, \ldots, \Theta\}$, the server computes:

$$z_i = [c^* \cdot y_i]_2$$

keeping only $n = \lceil \log \theta \rceil + 3$ bits of precision. The server outputs c^* and $\vec{z} = \langle z_1, \ldots, z_\Theta \rangle$.

Deriving the Squashed Decryption: We need to compute $c^*/p \pmod 2$.

$$\frac{c^*}{p} \approx c^* \sum_{i \in S} y_i \pmod 2 = \sum_{i=1}^{\Theta} s_i (c^* y_i) \pmod 2 \approx \sum_{i=1}^{\Theta} s_i z_i \pmod 2$$

Modifying Evaluate and Decrypt

When the server finishes homomorphically evaluating a circuit, it holds a noisy ciphertext c^*. **Before returning it**, the server does post-processing in the clear.

Server Post-Processing: For $i \in \{1, \ldots, \Theta\}$, the server computes:

$$z_i = [c^* \cdot y_i]_2$$

keeping only $n = \lceil \log \theta \rceil + 3$ bits of precision. The server outputs c^* and $\vec{z} = \langle z_1, \ldots, z_\Theta \rangle$.

Deriving the Squashed Decryption: We need to compute $c^*/p \pmod 2$.

$$\frac{c^*}{p} \approx c^* \sum_{i \in S} y_i \pmod 2 = \sum_{i=1}^{\Theta} s_i (c^* y_i) \pmod 2 \approx \sum_{i=1}^{\Theta} s_i z_i \pmod 2$$

The New Decrypt Algorithm:

$$m' = \left[c^* - \left\lfloor \sum_{i=1}^{\Theta} s_i z_i \right\rceil \right]_2$$

The Shallow Decryption Circuit

Look at our new decryption equation: $m' = \left[c^* - \lfloor \sum_{i=1}^{\Theta} s_i z_i \rceil\right]_2$.

The Shallow Decryption Circuit

Look at our new decryption equation: $m' = \left[c^* - \lfloor \sum_{i=1}^{\Theta} s_i z_i \rceil \right]_2$.

Why is this a mathematical triumph?

- The server has already provided the rational numbers z_i.
- The secret key bits s_i are strictly 0 or 1.
- Our only job is to sum up numbers. Because $\vec{s}$ is sparse, only θ of them are non-zero!

The Shallow Decryption Circuit

Look at our new decryption equation: $m' = \left[c^* - \lfloor \sum_{i=1}^{\Theta} s_i z_i \rceil \right]_2$.

Why is this a mathematical triumph?

- The server has already provided the rational numbers z_i.
- The secret key bits s_i are strictly 0 or 1.
- Our only job is to sum up numbers. Because $\vec{s}$ is sparse, only θ of them are non-zero!

Lemma (4.1: Elementary Symmetric Polynomials)

We use the classic "three-for-two" trick to reduce the sum. The i-th bit of the Hamming weight of a binary vector $\vec{\sigma}$ can be expressed as a polynomial of degree exactly 2^i using the 2^i-th elementary symmetric polynomial $e_{2^i}(\vec{\sigma})$.

Bounding the Degree

Because we only need $\lceil \log \theta \rceil + 3$ bits of precision after the binary point, the maximum degree required to compute the binary sum and rounding is strictly bounded by:

$$\text{Degree} \leq 64\theta \log^2 \theta$$

Bounding the Degree

Because we only need $\lceil \log \theta \rceil + 3$ bits of precision after the binary point, the maximum degree required to compute the binary sum and rounding is strictly bounded by:

$$\text{Degree} \leq 64\theta \log^2 \theta$$

Intuition

By forcing the evaluator to pre-compute the partial products z_i using the hint y_i, we squashed the massively deep integer division circuit down to a simple, sparse subset sum.

The Grand Finale: Bootstrapping Achieved

By setting $\theta = \lambda$, the degree of our squashed decryption polynomial becomes $\mathcal{O}(\lambda \log^2 \lambda)$.

The Grand Finale: Bootstrapping Achieved

By setting $\theta = \lambda$, the degree of our squashed decryption polynomial becomes $\mathcal{O}(\lambda \log^2 \lambda)$.

Theorem (6.2: Bootstrapping Achieved)

If we set our secret key length $\eta = \rho \cdot \Theta(\lambda \log^2 \lambda)$, the base SHE scheme's noise budget can comfortably handle polynomials of degree $\mathcal{O}(\lambda \log^2 \lambda)$.

The Grand Finale: Bootstrapping Achieved

By setting $\theta = \lambda$, the degree of our squashed decryption polynomial becomes $\mathcal{O}(\lambda \log^2 \lambda)$.

Theorem (6.2: Bootstrapping Achieved)

If we set our secret key length $\eta = \rho \cdot \Theta(\lambda \log^2 \lambda)$, the base SHE scheme's noise budget can comfortably handle polynomials of degree $\mathcal{O}(\lambda \log^2 \lambda)$.
Because the degree of the squashed decryption circuit is strictly less than the maximum degree of the permitted polynomials, we have:

$$\mathcal{D}_{\mathcal{E}} \subset \mathcal{C}_{\mathcal{E}}$$

Therefore, the scheme is bootstrappable.

The Grand Finale: Bootstrapping Achieved

By setting $\theta = \lambda$, the degree of our squashed decryption polynomial becomes $\mathcal{O}(\lambda \log^2 \lambda)$.

Theorem (6.2: Bootstrapping Achieved)

If we set our secret key length $\eta = \rho \cdot \Theta(\lambda \log^2 \lambda)$, the base SHE scheme's noise budget can comfortably handle polynomials of degree $\mathcal{O}(\lambda \log^2 \lambda)$.
Because the degree of the squashed decryption circuit is strictly less than the maximum degree of the permitted polynomials, we have:

$$\mathcal{D}_{\mathcal{E}} \subset \mathcal{C}_{\mathcal{E}}$$

Therefore, the scheme is bootstrappable.

We have achieved Fully Homomorphic Encryption using nothing but elementary modular arithmetic over the integers.

References

- Marten van Dijk, Craig Gentry, Shai Halevi, and Vinod Vaikuntanathan. *Fully Homomorphic Encryption over the Integers*. Eurocrypt, 2010.
- Craig Gentry. *A Fully Homomorphic Encryption Scheme*. PhD Thesis, Stanford University, 2009.
- J. Boyar, R. Peralta, and D. Pochuev. *On the multiplicative complexity of boolean functions over the basis ($\wedge$, $\oplus$, 1)*. Theor. Comput. Sci., 2000.